

Self-Reflective Multi-Agent Systems for Personalized Productivity Optimization

Bao Gia Khuong

Bellini College of Artificial Intelligence, Cybersecurity and Computing
University of South Florida
Tampa, FL, USA
giabaokhuong@usf.edu

Abstract

Although large language models (LLMs) have demonstrated significant capabilities in task automation, their application in personal productivity optimization remains largely limited to static scheduling and generic advice. In this paper, we introduce a novel, closed-loop multi-agent system designed to iteratively analyze, simulate, and adapt to individual productivity behaviors. Our framework comprises five specialized agents—Observer, Planner, Simulator, Evaluator, and Adapter—that collaborate to process historical productivity logs. Unlike existing systems that immediately recommend generated strategies, our architecture features a Simulator agent that projects the impact of proposed interventions on key metrics (e.g., completion rate, energy alignment) using historical baselines. An Evaluator agent then ranks these strategies, while an Adapter agent continuously tunes the system’s evaluation weights based on the delta between predicted and actual real-world outcomes. We demonstrate the efficacy of this observe-plan-simulate-evaluate-adapt loop through a controlled case study, highlighting the system’s ability to reduce scheduling conflicts and optimize deep work allocation while maintaining high user-energy alignment.

1 Introduction

Personal productivity optimization is a complex, highly individualized challenge. Traditional software solutions, such as calendar applications and habit trackers, rely on static algorithms that require users to manually identify their own inefficiencies. Recent advancements in AI have enabled the development of autonomous agents capable of reasoning, planning, and tool use. However, when applied to personal scheduling, current approaches often act as single-pass generators, providing generalized advice without evaluating the feasibility of their suggestions against a user’s actual habits.

To address this, we propose a multi-agent framework that treats personal productivity as an iterative

optimization problem. Drawing inspiration from self-reflective agent architectures, our system does not merely suggest changes; it simulates their potential outcomes before recommending them, and then learns from the real-world results.

The core contribution of this project is a fully functional, five-stage agentic pipeline:

1. Data-driven observation of inefficiencies.
2. Targeted strategy generation.
3. "What-if" simulations grounded in historical baselines.
4. Weighted evaluation of simulated outcomes.
5. A continuous feedback loop that adjusts system priorities based on real-world adaptation.

2 Related Work

- **LLM Agents and Reasoning:** Recent frameworks like ReAct (1) and Reflexion (2) have shown that LLMs perform better when prompted to think step-by-step and reflect on past failures. Our system applies this concept at a macro level, using a dedicated Adapter agent to reflect on the performance of the entire pipeline. Additionally, recent work has demonstrated that LLMs show promise in combinatorial scheduling tasks (5), motivating their application in broader productivity optimization domains.
- **Multi-Agent Collaboration:** Splitting complex tasks among specialized agents reduces hallucination and improves output structure. By compartmentalizing the roles (e.g., separating the "Planner" from the "Evaluator"), the system maintains objectivity. The Planner is free to brainstorm, while the Evaluator acts as a strict filter.
- **Synthetic Data for Benchmarking:** Evaluating personalized AI systems is difficult due to privacy constraints surrounding real user

data. Similar to recent trends in NLP (4), we utilize a synthetically generated longitudinal dataset to safely benchmark the agents’ ability to detect patterns and improve metrics over time.

3 System Architecture

The system is built in Python, utilizing the Google Gemini API (specifically the gemini-2.5-flash-lite model) to power the reasoning of five distinct agents. The agents do not communicate via conversational text; instead, they pass structured JSON schemas (enforced via the Pydantic library) to ensure reliable pipeline execution and deterministic outputs.

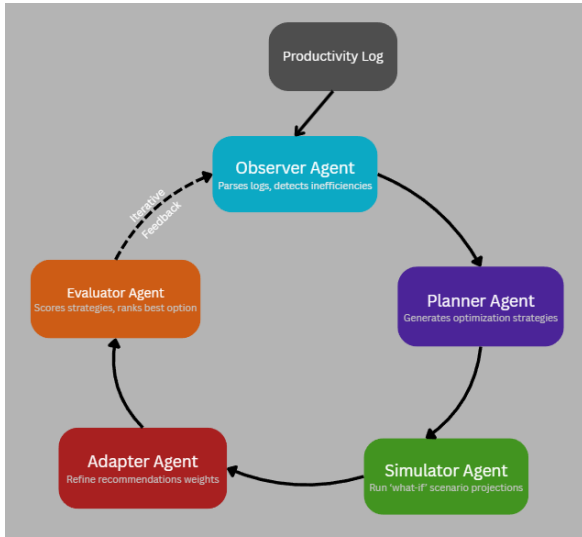


Figure 1: The observe-plan-simulate-evaluate-adapt loop of the multi-agent productivity optimization system.

3.1 The Observer Agent

The cycle begins with the Observer, which acts as the data ingestion layer. It processes the user’s `productivity_logs.csv`, computing baseline statistics such as completion rates, energy alignment (a 1-5 scale of how much energy the user had during a task), and time-of-day efficiency. The LLM then reviews these raw statistics to generate a list of concrete "Insights" and actionable "Inefficiencies" (e.g., "Deep work scheduled in the late afternoon consistently results in low energy and a 40% completion rate").

3.2 The Planner Agent

The Planner receives the Observer’s inefficiencies and generates 2-3 specific optimization strategies.

Crucially, these are not vague suggestions. Each strategy includes a specific rationale, a list of concrete implementation steps, and a qualitative prediction of how it will increase completion rates and energy alignment.

3.3 The Simulator Agent (The “What-If” Engine)

Alongside projected metrics, the Simulator outputs two additional structured fields: a confidence tier and a `confidence_reason`. The confidence tier is a constrained categorical value — either low, medium, or high — that the LLM assigns based on how strongly the historical baseline supports the proposed intervention’s underlying assumptions. The accompanying `confidence_reason` field provides a natural language justification for this rating, explicitly referencing the key assumptions listed in the simulation output. For example, a strategy may receive a medium rating when its rationale is logically sound but the historical data does not provide strong direct evidence that the projected metric gains will materialize. This pairing ensures that the Evaluator agent receives not only a scalar signal but also an interpretable explanation, enabling it to make more nuanced ranking decisions rather than treating all projections as equally reliable.

3.4 The Evaluator Agent

The Evaluator acts as the decision-maker. It receives the simulated projections and scores each strategy using a weighted formula. The default evaluation formula prioritizes task completion and deep work, calculated as:

$$WeightedScore = (0.35 \times C_{gain}) + (0.25 \times E_{gain}) + (0.25 \times D_{gain}) + (0.15 \times R_{conflicts}) \quad (1)$$

(Where C is Completion Rate, E is Energy Alignment, D is Deep Work Hours, and R is Conflict Reduction).

The Evaluator considers the implementation difficulty and the Simulator’s confidence level to output a final ranked list and a single “Quick Win” recommendation for the user.

3.5 The Adapter Agent

To make the system truly agentic, it must learn. After a cycle concludes, the user attempts the recommended strategy and inputs their actual observed

metrics. The Adapter compares the Simulator’s predicted metrics against the actual metrics. Based on the delta (the prediction gap), the Adapter writes "Lessons Learned" and dynamically updates the scoring weights (the 0.35, 0.25, etc., in the Evaluator’s formula) for the next cycle. If a strategy failed to improve energy alignment as predicted, the Adapter might increase the weight of E_{gain} for future evaluations.

3.6 User Interface and Implementation

To demonstrate the system’s capabilities and make the multi-agent collaboration transparent to the end-user, we implemented a highly interactive web interface using the Streamlit framework.

Rather than executing as a silent backend script, the dashboard visualizes the cognitive pipeline in two distinct phases:

- **Strategy Generation Pipeline:** A step-by-step execution view where the user can watch the Observer extract insights, the Planner draft interventions, the Simulator project metrics, and the Evaluator rank the final recommendations.
- **Adapter Feedback Loop:** An interactive form where the user can input their actual, real-world metrics after trialing a strategy. This triggers the Adapter agent to process the reality gap and dynamically update the system’s scoring weights on screen.

This frontend interface effectively showcases the role of multi-agent systems in problem-solving by exposing the intermediate structured data passing between agents, ensuring the system is both functional and interpretable.

As illustrated in Figure 2 and 3, the dashboard provides a high-level overview of the optimization cycle, exposing the underlying reasoning of each specialized agent

4 Experimental Setup

To validate the architecture, we tested the pipeline on a synthetically generated dataset designed to mimic a typical undergraduate intern’s schedule. Evaluating personalized AI systems is often difficult due to privacy constraints surrounding real user data (4). By generating synthetic data, we can safely benchmark the agents’ ability to detect patterns and improve metrics over time.

4.1 Dataset Characteristics

As shown in Table 1, the dataset contains tabular data detailing tasks, categories (e.g., deep_work, admin, learning), start and end times, completion status (a boolean value), and self-reported energy levels on a scale of 1 to 5.

Crucially, the dataset was engineered to contain hidden, systemic inefficiencies. These include frequent scheduling overlaps and a distinct negative correlation between late-afternoon deep work and task completion. The system was run through an initial generation pipeline (Observer → Planner → Simulator → Evaluator) to observe its ability to correctly identify these engineered inefficiencies from the raw CSV data without any explicit human prompting or hints.

5 Results and Case Study

To evaluate the end-to-end functionality of the framework, we executed a complete observe-plan-simulate-evaluate-adapt cycle using the synthetic dataset. The agents successfully parsed the raw data, generated insights, and iteratively arrived at a dynamically weighted recommendation.

5.1 Phase 1: Observation and Planning

During the initial ingestion phase, the **Observer** successfully isolated key temporal and behavioral patterns without human guidance. It calculated an overall task completion rate of 75.0% across 112 logged tasks, with an average energy level of 3.53. More importantly, it identified a stark contrast in time-of-day efficiency: the early morning (8:00–10:00 AM) slot boasted a 90.0% completion rate, with tasks at energy levels 4 and 5 achieving 85.4% and 92.3% completion rates respectively. Conversely, the late afternoon (4:00 PM onwards) fell to a 58.3% completion rate and a mean energy level of 2.58. The Observer also flagged systemic scheduling overlaps involving meetings and collaborative tasks, and noted that the learning category exhibited the lowest completion rate among all task categories at 68.8%, suggesting inadequate time allocation for cognitively demanding study tasks.

5.2 Phase 2: Simulation and Evaluation

Based on these targeted inefficiencies, the **Planner** drafted three interventions: (1) *Prioritize High-Energy Mornings for Deep and Learning Tasks*, (2) *Shift Afternoon Tasks to Lighter Activities*, and (3) *Implement a Strict No-Overlap Policy for Deep*

date	task_name	category	start_time	end_time	energy_level	completed
2025-01-09	Practice LeetCode	learning	08:45	09:30	4	True
2025-01-09	Code feature implementation	deep_work	10:15	11:00	4	False
2025-01-09	Team standup	meetings	12:15	13:45	3	True
2025-01-09	System architecture	deep_work	13:30	15:00	3	True
2025-01-09	Weekly planning	admin	16:15	17:15	2	True
2025-01-09	Documentation update	admin	17:00	18:30	3	True
2025-01-10	Model training	deep_work	08:30	09:30	4	True
2025-01-10	Data analysis	deep_work	10:30	12:00	3	True
2025-01-10	Model training	deep_work	13:15	14:00	2	True
2025-01-10	Code review session	meetings	15:45	16:30	3	True

Table 1: Sample rows from the synthetic `productivity_logs.csv` dataset. The data was engineered to contain hidden patterns, such as a severe drop in task completion when complex `deep_work` is scheduled in the late afternoon.

Work and Meetings. These were passed to the **Simulator** to project real-world impacts.

The Simulator projected the following outcomes for each strategy:

The Simulator assigned *Prioritize High-Energy Mornings for Deep and Learning Tasks* a HIGH confidence rating, grounding its reasoning in the observation that the baseline data strongly supported mornings as the most productive period, with significantly higher completion rates and energy levels. The second strategy received only a MEDIUM confidence rating, as shifting afternoon tasks does not guarantee a completion boost for those tasks and its overall impact was deemed moderate. The third strategy also received a HIGH confidence rating, as eliminating direct conflicts between deep work and meetings represents a clear logistical improvement addressing a known source of inefficiency.

The **Evaluator** processed these projections and ranked the strategies at 6.1/10, 2.7/10, and 1.4/10 respectively, officially designating *Prioritize High-Energy Mornings* as the top recommendation. The Evaluator noted that the strategy’s projected gains in completion rate (+8.0%) and deep work hours (+0.7h/wk) — achieved with high simulator confidence — heavily outweighed the moderate implementation difficulty of managing team scheduling. To bridge the gap between theory and execution, the Evaluator generated an actionable “Quick Win”: “*Identify your typical high-energy period (likely morning) and block out at least one hour for deep work tomorrow.*”

5.3 Phase 3: The Adapter Feedback Loop

To validate the system’s ability to learn, we simulated a completed week where the user imple-

mented the top-ranked strategy and fed the actual resulting metrics back into the system. The user reported a completion rate of 89.0%, an average energy of 4.40, 12.0 deep work hours per week, and 1 remaining scheduling conflict — exceeding the Simulator’s projections on completion rate and deep work hours, while energy alignment showed slightly less improvement than predicted.

Processing the delta between the Simulator’s predictions and these actual outcomes, the **Adapter** concluded that the strategy had successfully boosted completion rates and deep work hours beyond expectations, but that scheduling conflicts, while reduced, had not been fully eliminated. It therefore identified energy alignment as the focus area for the next cycle. Consequently, the Adapter updated the Evaluator’s scoring weights: `completion_rate_gain` was raised to 0.40, `energy_alignment_gain` stayed at 0.25, `deep_work_hours_gain` to 0.3, and `conflict_reduction` was deprioritized to 0.05 — reflecting that conflicts are no longer the primary bottleneck. These updated weights force the Planner and Evaluator to focus on sustainable energy management and task execution depth in the subsequent optimization cycle.

6 Discussion and Limitations

While the framework successfully demonstrates complex multi-agent collaboration, there are notable limitations to the current architecture that must be addressed in future iterations.

Hallucination in Simulation: The Simulator agent relies on LLM reasoning to project quantitative metrics. While grounding its prompts in his-

Strategy	Completion	Avg Energy	Deep Work	Conflicts/wk
Prioritize High-Energy Mornings	83.0% ($\uparrow 8.0\%$)	4.0 ($\uparrow 0.4$)	9.5h ($\uparrow 0.7$)	1 ($\downarrow 1$)
Shift Afternoon Tasks	79.0% ($\uparrow 4.0\%$)	3.4 ($\downarrow 0.2$)	8.5h ($\downarrow 0.3$)	2 (± 0)
Strict No-Overlap Policy	78.0% ($\uparrow 3.0\%$)	3.7 ($\uparrow 0.1$)	8.7h ($\downarrow 0.1$)	0 ($\downarrow 2$)

Table 2: Simulated metric projections for each Planner strategy.

torical baselines significantly curtails drift, LLM-generated outputs remain susceptible to hallucination (3), and the projected metrics remain probabilistic estimates rather than strict mathematical proofs.

Data Dependency and Cold Starts: The system relies heavily on the user consistently tracking their time and energy levels. If the input data is sparse or inaccurate, the Observer cannot generate meaningful insights. The system currently lacks a “cold start” mechanism for new users without extensive historical logs.

Synthetic Limitations: While the synthetic dataset effectively validates the software architecture and agentic communication, a real-world longitudinal study with actual human users is required to prove that the Adapter agent’s dynamic weight adjustments lead to sustained productivity increases over extended periods.

7 Conclusion

We presented a self-reflective, multi-agent system capable of closed-loop productivity optimization. By moving away from single-pass generation and introducing a dedicated simulation and adaptation phase, the system provides highly personalized, data-grounded recommendations. The structured JSON communication between the Observer, Planner, Simulator, Evaluator, and Adapter ensures pipeline stability and demonstrates the immense potential of agentic workflows in personalized software applications. Future work will focus on integrating real-time calendar APIs and replacing synthetic logs with live user studies to further refine the Adapter’s learning mechanism.

8 Acknowledgement

This project was completed solely by Bao Gia Khuong (100%). All components of the final projects including system architecture design, agent implementation, dataset generation, experimental evaluation, and manuscript writing were

conceived and executed independently.

AI Assistant Disclosure

During the development of this project, AI assistants (Google Gemini, Cursor) were utilized to assist with software architecture ideation, code debugging, and LaTeX formatting. I extensively reviewed, edited, and take full responsibility for the final code execution and manuscript content.

References

- [1] Yao, S. et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] Shinn, N. et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [3] Huang, L. et al., “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [4] Ramesh, K. et al., “Evaluating Differentially Private Synthetic Data Generation in High-Stakes Domains,” *Findings of EMNLP 2024*, 2024.
- [5] Abgaryan, H. et al., “LLMs can Schedule,” *arXiv preprint arXiv:2408.06993*, 2024.

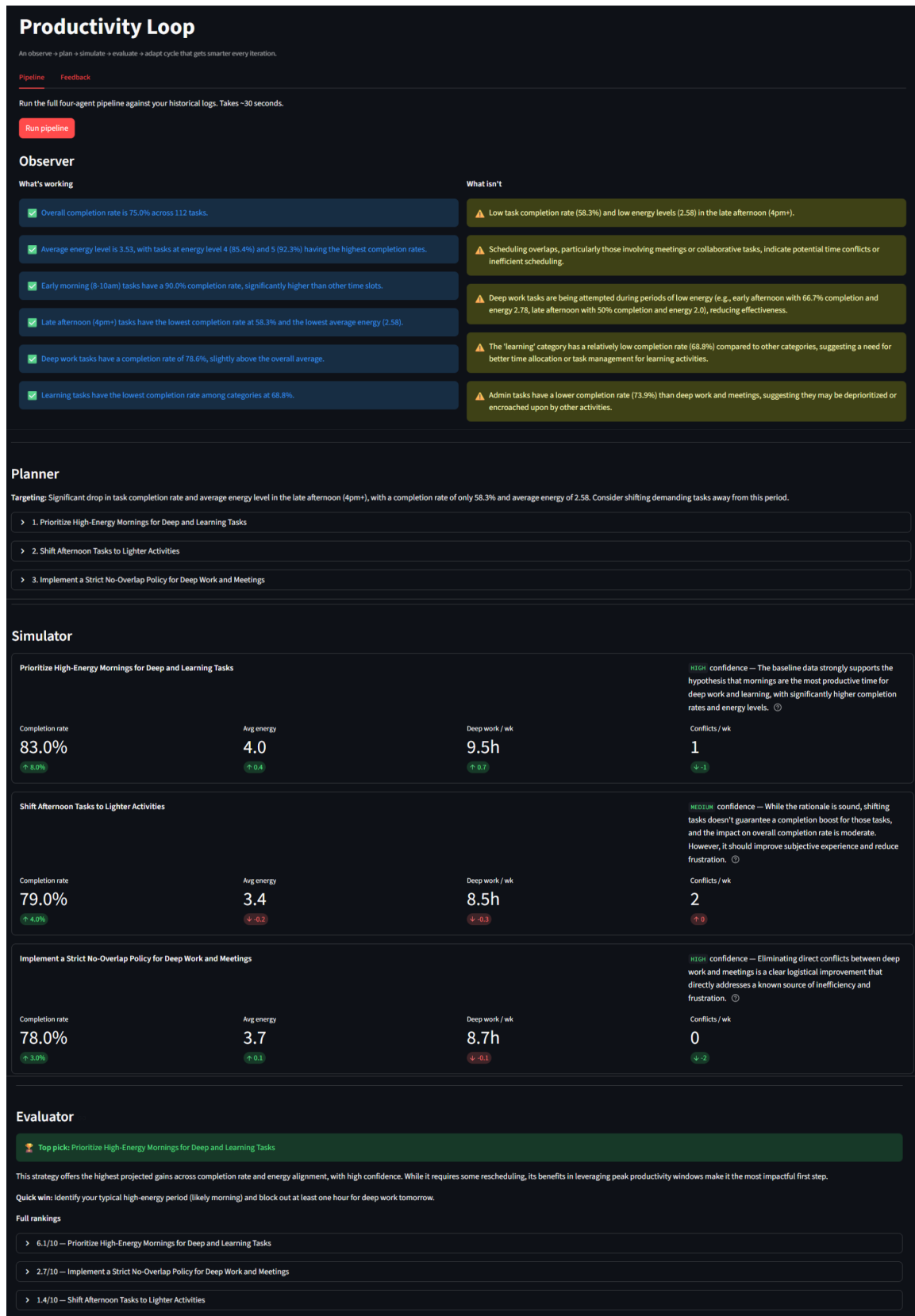


Figure 2: The Streamlit dashboard interface. The UI visualizes the multi-agent reasoning chain in real-time, allowing the user to inspect intermediate agent outputs, projected metrics from the Simulator, and provide feedback to the Adapter loop.

Pipeline

Feedback

You tried the top strategy for a week. Enter what actually happened and the Adapter will reconcile the gap and update the system's priors.

Strategy you trialled

Prioritize High-Energy Mornings for Deep and Learning Tasks

Actual results after one week

Completion rate (%)	Avg energy (1-5)	Deep work (hrs/wk)	Conflicts / wk
89.00	4.40	12.00	1

Run Adapter

This cycle's strategy successfully boosted completion rates and deep work hours, exceeding predictions. We will slightly increase the weight for deep work hours and completion rate gains, while reducing the weight for conflict reduction, as this area showed no issues.

Lessons learned

- The strategy to prioritize high-energy mornings for deep and learning tasks was effective, leading to higher completion rates and more deep work hours than predicted.
- While energy alignment was slightly better than predicted, there's still room for improvement in fully optimizing energy levels throughout the day.
- The scheduling conflicts remained stable, indicating that the current scheduling approach is robust but doesn't necessarily need a higher weight.

Next cycle focus: energy_alignment_gain

Updated system weights

```
{
  "completion_rate_gain": 0.4
  "energy_alignment_gain": 0.25
  "deep_work_hours_gain": 0.3
  "conflict_reduction": 0.85
}
```

Figure 3: The Feedback to the Adapter loop interface.